

Actividad 2

UD3

Pavel Miron

Índice

Contenido

Creación de subredes.....	2
La configuración de tus contenedores, incluyendo un esquema gráfico del despliegue	2
Solución a las preguntas sobre interconexión de contenedores, incluyendo posibles modificaciones a incluir.....	3

Creación de subredes

Creamos dos subredes que los vamos a utilizar.

```
daw2@COMPUS040 MINGW64 ~
$ docker network create --subnet 192.18.0.0/24 public-net
51580b62e583feaa10633e20bca51684d29a22f70b07818fcdad86be6bfc64bf

daw2@COMPUS040 MINGW64 ~
$ docker network create --subnet 192.19.0.0/24 private-net
7037271a7d6a7c1d88ceb90649eab0ac39e0ab324c0d68256d3b818ace831b85
```

Comprobamos que se han creado con “docker network ls”

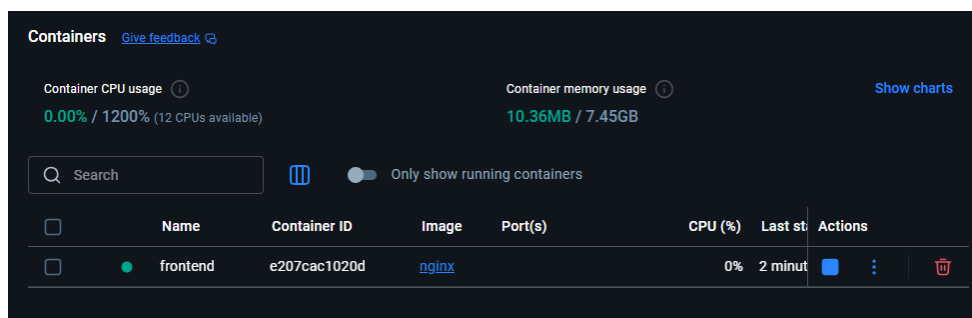
```
daw2@COMPUS040 MINGW64 ~
$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
4c912e67e6d9        bridge              bridge              local
c22b17326617        host                host                local
bdcba9a294a         none                null                local
7037271a7d6a        private-net         bridge              local
51580b62e583        public-net          bridge              local
```

La configuración de tus contenedores, incluyendo un esquema gráfico del despliegue

Creamos el contenedor para front end en el comando “docker run -d --name frontend --network public-net nginx”

```
daw2@COMPUS040 MINGW64 ~
$ docker run -d --name frontend --network public-net nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
1733a4cd5954: Pull complete
114e699da838: Pull complete
5b5fa0b64d74: Pull complete
ee3a09d2248a: Pull complete
7382b41547b8: Pull complete
9ee60c6c0558: Pull complete
5b219a92f92a: Pull complete
11488ed04caf: Download complete
adeb5aba46ee: Download complete
Digest: sha256:fb01117203ff38c2f9af91db1a7409459182a37c87cced5cb442d1d8fcc66d19
Status: Downloaded newer image for nginx:latest
e207cac1020d1a117c448c6d62de1dbb393aea8e44fec9d1d7f695e2b0bef504
```

Comprobamos que esta en docker.



Hacemos lo mismo con back y comprobamos las ips de los dos contenedores

```
daw2@COMPUS040 MINGW64 ~  
$ docker run -d --name backend --network private-net nginx  
bf79e376bac28a84b39ffec94f99b29059375a5fe6fbce61c3884d824c1980d6
```

Containers [Give feedback](#)

Container CPU usage ⓘ 0.00% / 1200% (12 CPUs available) Container memory usage ⓘ 20.94MB / 7.45GB [Show charts](#)

Search ☐ Only show running containers

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last state	Actions
☐	frontend	e207cac1020d	nginx		0%	18 minutes ago	
☐	backend	bf79e376bac2	nginx		0%	2 minutes ago	

Ahora para comprobar que ips tiene cada ejecutamos el comando “docker inspect frontend | grep IPAddress” y para back “docker inspect backend | grep IPAddress”

```
daw2@COMPUS040 MINGW64 ~  
$ docker inspect backend | grep IPAddress  
    "SecondaryIPAddresses": null,  
    "IPAddress": "",  
    "IPAddress": "192.19.0.2",
```

```
daw2@COMPUS040 MINGW64 ~  
$ docker inspect frontend | grep IPAddress  
    "SecondaryIPAddresses": null,  
    "IPAddress": "",  
    "IPAddress": "192.18.0.2",
```

Vemos que cada contenedor tiene la ip correspondiente.

Ahora hacemos el ping para comprobar la conexión entre ellos.

Pero antes de hacer ping se tiene que instalar el comando “ping” porque el Nginx por defecto no lo tiene accedemos “docker exec -it frontend sh” y introducimos el siguiente comando “apt install -y iputils-ping”

Antes!

```
daw2@COMPUS040 MINGW64 ~  
$ docker exec -it frontend sh  
# ping 192.19.0.2  
sh: 1: ping: not found  
# exit
```

Después de instalar ping!

```
# ping 192.19.0.2  
PING 192.19.0.2 (192.19.0.2) 56(84) bytes of data.  
|
```

Vemos que al hacer el ping no se pueden ver (no reciben datos está parado). Porque en Docker las redes son distintas y no se pueden comunicar entre sí.

Solución a las preguntas sobre interconexión de contenedores, incluyendo posibles modificaciones a incluir

La solución mas simple es conectar frontend a la red privada con “docker network connect private-net frontend”. Mas abajo podemos ver que ahora el frontend esta dentro de la red del back.

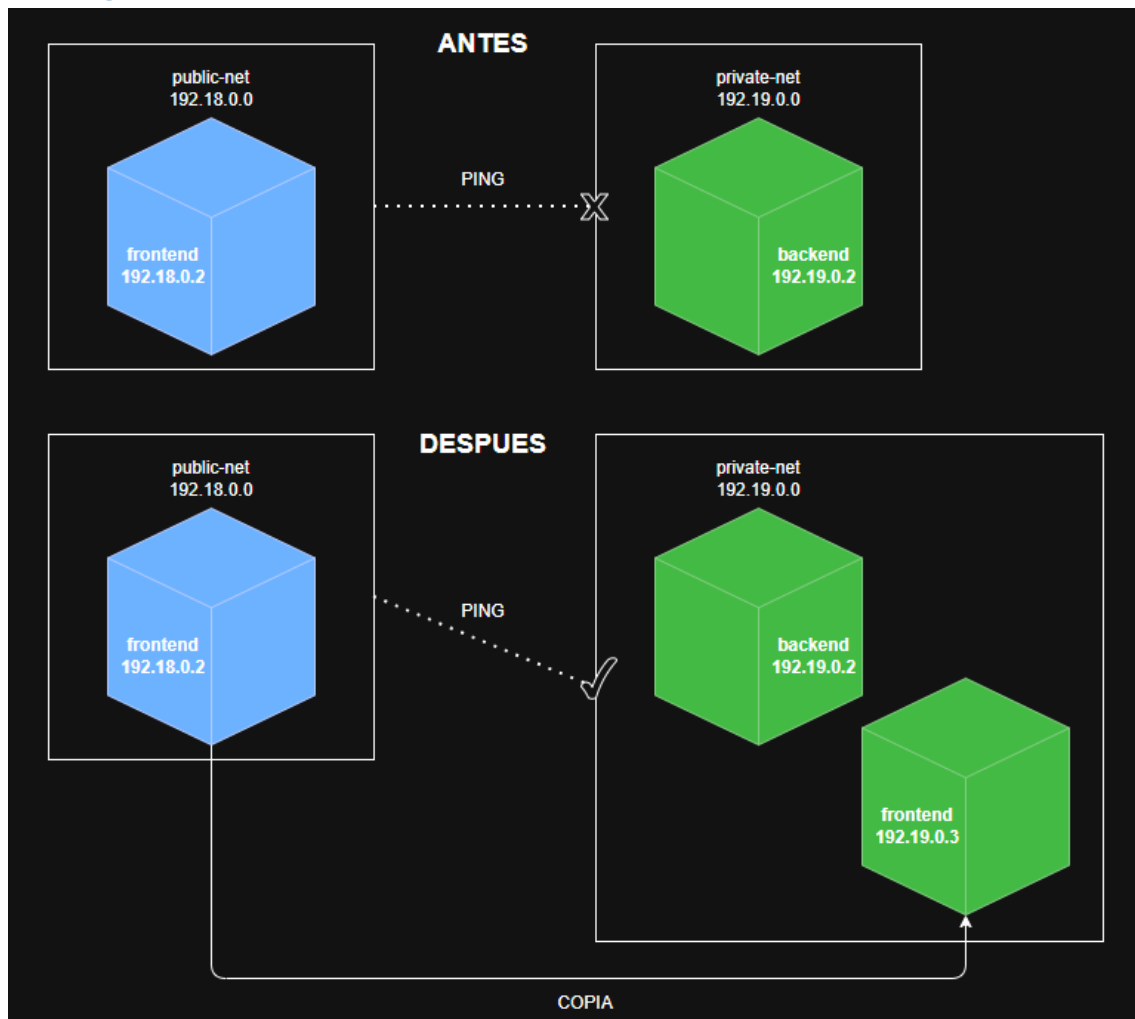
```
daw2@COMPUS040 MINGW64 ~  
$ docker network connect private-net frontend  
  
daw2@COMPUS040 MINGW64 ~  
$ docker inspect frontend | grep IPAddress  
    "SecondaryIPAddresses": null,  
    "IPAddress": "",  
    "IPAddress": "192.19.0.3",  
    "IPAddress": "192.18.0.2",  
  
daw2@COMPUS040 MINGW64 ~  
$
```

Ahora comprobamos hacer el ping y si recibe los datos.

```
daw2@COMPUS040 MINGW64 ~  
$ docker exec -it frontend sh  
# ping 192.19.0.2  
PING 192.19.0.2 (192.19.0.2) 56(84) bytes of data.  
64 bytes from 192.19.0.2: icmp_seq=1 ttl=64 time=0.480 ms  
64 bytes from 192.19.0.2: icmp_seq=2 ttl=64 time=0.103 ms  
64 bytes from 192.19.0.2: icmp_seq=3 ttl=64 time=0.111 ms  
64 bytes from 192.19.0.2: icmp_seq=4 ttl=64 time=0.378 ms  
64 bytes from 192.19.0.2: icmp_seq=5 ttl=64 time=0.144 ms  
64 bytes from 192.19.0.2: icmp_seq=6 ttl=64 time=0.059 ms  
^C  
--- 192.19.0.2 ping statistics ---  
6 packets transmitted, 6 received, 0% packet loss, time 5117ms  
rtt min/avg/max/mdev = 0.059/0.212/0.480/0.157 ms  
#
```

Como podemos ver los datos están recibidos bien y hay conexión entre contenedores, porque están en misma red. El front esta dentro de dos redes.

Esquema gráfico del despliegue antes de corregir y después de corregir



En esta esquema se puede ver que después de mover el contenedor FRONTEND de la red public a private no deja de estar en suya ref., pero después de moverse toma también la ip de la red private que nos da la posibilidad que se comuniquen entre si.